Uncertainty Estimation for Long-Form Retrieval Augmented Generation (RAG)

Muhammad Ali
ali.mahmood@ru.nl(s1175670)
Radboud University
Nijmegen, Netherlands

Rijk Kregting
rijk.kregting@ru.nl(s1044740)
Radboud University
Nijmegen, Netherlands

Adith Shivakumar
adith.shivakumar@ru.nl(s1166398)
Radboud University
Nijmegen, Netherlands

Abstract

Retrieval-Augmented Generation (RAG) systems combine document retrieval with the LLMs to improve factual accuracy, yet they still produce incorrect or unsupported output when relevant information is missing. This limitation highlights the need for uncertainty estimation methods that allow RAG systems to indicate when their outputs should not be blindly trusted. Previous research has shown that uncertainty methods do not perform well in a short-form, single answer RAG setting[6]. In this project, we investigate how uncertainty estimation performs in a long-form RAG setup. We implemented one white-box, and two black box methods, MARS, Eccentricity and the novel SEEW. Our results show that these uncertainty methods do not perform well in a long-form RAG setup.

1 Introduction

Retrieval-Augmented Generation (RAG) has emerged as a fundamental paradigm for expanding LLMs beyond their static training limitations. In these systems, the quality of the final answer is heavily dependent on two components: the relevance of the retrieved documents and the reliability of the response generated by the language model. Although RAG improves factual accuracy compared to standalone LLMs, it still suffers from uncertainty, especially when the retrieved evidence is weak or ambiguous. This creates a need to add an uncertainty estimation method to help us identify when a system is likely to produce incorrect or unsupported answers. Therefore, developing an uncertainty estimation mechanism is essential to produce safer and more transparent RAG pipelines.

The aim of our project is not only to identify when the model is uncertain but also to investigate whether uncertainty estimators (such as MARS[1] and Eccentricity[4]) correlate meaningfully with long-form factuality, as measured by SAFE[7]. Since SAFE assesses the correctness at the claim level, it provides a reliable truth against which we can compare the uncertainty scores. By analyzing both retrieval-based and generation-based indicators, we can evaluate whether higher levels of uncertainty indicate a higher likelihood of factual errors. This will help us check how effectively each uncertainty method captures actual model reliability, rather than simply internal variability. Finally, we aim to determine whether these uncertainty measures can serve as practical predictors of factual correctness in Retrieval-Augmented Generation (RAG) systems.

In general, our approach contributes to improving the reliability, interpretability, and user trustworthiness of RAG-based applications. In the following sections, we outline the motivation for uncertainty estimation, and present our proposed method along with its evaluation.

2 Background

2.1 Retrieval-Augmented Generation (RAG)

The RAG pipeline consists of two components:

(1) **Retriever:** Which helps in retrieving k relevant text passages from a large corpus.
(2) **Generator:** This produces a long-form answer formatted on retrieved documents.

2.2 Long-Form Evaluation with SAFE

SAFE[7] is a long-form factuality evaluation method that combines LLM prompting with information retrieval. It first decomposes a generated answer into atomic factual claims and rewrites them as standalone statements. In the original SAFE pipeline, each claim is verified by querying Serper/Google Search and then prompting an LLM to judge whether the retrieved results support the claim.

In our setup, we do not use the Google/Serper API. Instead, we replace the search step with our own retriever (BM25 over a Wikipedia-based corpus) followed by cross-encoder reranking. The top reranked passages are used as evidence, and the verifier LLM decides whether they support each claim. SAFE also proposes an F1-style metric that penalizes overly short but factual answers using a target-facts hyperparameter K.

3 Uncertainity Estimations Methods

3.1 White-box UE

White-box uncertainty estimation (UE) methods utilise internal model components, such as weights, activations, or token-level logits, to compute an uncertainty estimate. This contrasts with black-box approaches that rely solely on inputs and outputs. Below some of these methods are given of which MARS is used in this project.

3.2 Core White-Box Methods

Monte Carlo Dropout (MCD)[2] treats dropout as variational inference: this enables dropout at test time, run T=50 forward passes, and also computes predictive variance $\mathrm{Var}[\hat{y}] = \frac{1}{T}\Sigma_t(\hat{y}t - \bar{y})^2$.

MARS (Meaning-Aware Response Scoring)[1] changes the usual sequence probability so that important words influence the score more than unimportant ones.

The standard (length-aware) sequence score is replaced by

$$\bar{P}(s \mid x, \theta) = \prod_{l=1}^{L} P(s_l \mid s_{<l}, x; \theta)^{w(s,x,L,l)} \quad (1)$$

, where

- $s = (s_1, \ldots, s_L)$is the generated answer,
- x is the question/context,
- $P(sl \mid s < l, x; \delta)$ is the model's probability for token sl,
- w(s,x,L,l) is a weight telling how important token l is.

The weight is defined as

$$w(s, x, L, l) \coloneqq \frac{1}{2L} + \frac{u(s, x, l)}{2} \tag{2}$$

where L is the answer length and u(s,x,l)u(s,x,l) is the importance function.

So if low-probability tokens are also significant (high u), the overall score

$$\bar{P}(s \mid x, \theta) \tag{3}$$

becomes low → high uncertainty.

If the important tokens have a high probability, the score is high → the model is confident.

3.3 Black-box UE:

Black-box uncertainty estimation (UE) methods treat models as opaque systems, using only inputs and outputs, where weights, activations, or computations are hidden from users, to quantify epistemic and aleatoric uncertainty. These are best suited for proprietary APIs, such as GPT-4, or closed-source large language models in retrieval-augmented generation (RAG) pipelines.

3.4 Core Black-Box Methods

Semantic Entropy[3] quantifies dispersion across multiple sampled generations: generate T=20 responses using temperature sampling, embed them with Sentence-BERT, and calculate the variance in the embedding space. High variance indicates uncertainty, making it straightforward and effective for detecting hallucinations.

Self-Consistency[9] samples K=10 outputs per query, checks agreement via majority vote or semantic similarity. Disagreement → high epistemic uncertainty. Computationally cheap baseline.

Eccentricity (ECC)[4] measures uncertainty as the average distance of response embeddings from their centroid. For black-box LLMs lacking direct embeddings, these are derived from the graph Laplacian $\hat{L}$'s k smallest eigenvectors $\mathbf{u}1,\ldots,\mathbf{u}kRB$. Each response rj's embedding is $\mathbf{v}j=[\mathbf{u}1,j,\ldots,\mathbf{u}k,j]$, with centroid $\mathbf{v}j=\mathbf{v}jB1j=1B\mathbf{v}j$. Uncertainty is then computed as:

$$U_{\mathrm{ECC}}(x) = \left\| \left[\mathbf{v}_1'^{\top}, \ldots, \mathbf{v}_B'^{\top} \right] \right\|_2 \tag{4}$$

4 Methodology

4.1 Dataset

The dataset used for evaluating uncertainty estimation methods and safe in RAG is the Factscore-BIO[5] dataset. Of this dataset 50 samples are randomly selected.

4.2 RAG

The RAG system contains of two important parts, the retriever and an LLM. The dataset behind the retriever is the wiki-18-corpus from Huggingface, https://huggingface.co/datasets/PeterJinGo/wiki-18-corpus. The retriever setup consists of a BM25 index + Reranker model. The LLM used is Qwen2.5 7B. The implementation of both the retriever and the LLM is explained in more detail below.

4.3 Retriever: BM25 + Cross-Encoder Reranking

The BM25 index is made using the Pyserini library. The index is made in a single command:

```
python -m pyserini.index.lucene \
  --collection JsonCollection \
  --input wiki_dump \
  --index indexes/wiki_dump \
  --generator DefaultLuceneDocumentGenerator \
  --threads 7 \
  --storePositions --storeDocvectors --storeRaw
```

In this retriever setup, the top 1000 retrieved documents using BM25 for a given query will be reranked using the **MS-Marco-MiniLM-L6-v2** model. The top three documents will be given as context to the LLM. This reranking process is made very easy with the Sentence Transformer package. The full code for the retriever is given below:

```
bm25 = LuceneSearcher('index/wiki_dump')
cross_encoder = \
  CrossEncoder(
    'cross-encoder/ms-marco-MiniLM-L6-v2'
  )
hits = bm25.search(query,k=1000)
docs = get_docs_from_hits(hits)
top_k = pl.DataFrame(
  cross_encoder.rank(
    query,docs,top_k=3,return_documents=True)
  )
```

4.4 LLM: Qwen2.5-7B, full & 4Bit quantized model

Our initial setup was only powered by an Nvidia RTX4070, with only 8GB of VRAM. The full precision model requires at least 17GB of VRAM. As our pipeline first was made on this quantized version the results for both models are given. The full precision model results are calculated on an Nvidia RTX5090.

Both models are loaded from Huggingface using the Transformer package. The quantized model has been quantized using the BitsAndBytes package is available at https://huggingface.co/unsloth/Qwen2.5-7B-Instruct-bnb-4bit while the full model is available at https://huggingface.co/Qwen/Qwen2.5-7B-Instruct.

4.5 Uncertainty Estimation & SAFE

The uncertainty estimation methods Mars and Eccentricity are implemented in the TruthTorchLM library[8]. This package also implements SAFE, as well as a technique to evaluate uncertainty methods in long-form generation called Claim Check Methods. As truth methods are mostly designed for short generation, they are likely not performing well on long-form generation. To account for this, the Claim Check Methods in TruthTorchLM, similarly to SAFE, decompose the long-form generation into individual claims.

Then the claim check method that is used in this project, QuestionAnswerGeneration, tries to form specific questions to which a decomposed claim should be an answer. The result of this is that truth methods can now be evaluated on a claim basis. Another advantage of this method is that it can capture the moment where a model is uncertain within a long-form generation. A clear disadvantage is that the uncertainty is not actually estimated for the generation of which an uncertainty score was requested.

The TruthTorchLM library was forked in this project and changed in several ways to better fit our setup. The following explains the three main changes:

(1) The removal of the SerperAPI calls in the SAFE implementation. We opted for our own retriever instead.
(2) The addition of context to the QuestionAnswerGeneration method. Without this, the model would not have any context when generating answers for an individual claim.

This fork is available at https://github.com/Rijkkie/TruthTorchLM/tree/main/src/TruthTorchLM.

The final code for measuring uncertainty in long-form and comparing it to SAFE is similar to the example code on the TruthTorchLM github. First, for the Factscore-BIO samples, context is already retrieved. After initializing both UE-methods, the next step is to calibrate the truth methods using the `calibrate_truth_method` function. This calibration is not strictly necessary and does not impact the correlation scores, but makes the UE results more readable. The final step is to call the `evaluate_truth_method_long_form` function which outputs both the AUROC and PRR score. Full code is available at https://github.com/Rijkkie/rag_ue_safe & https://github.com/alihawk/rag-uncertainty-longform.

4.6 SEEW (Semantic Entropy with Evidence Weighting) — Bonus Task

In addition to MARS and ECC, we implemented **SEEW (Semantic Entropy with Evidence Weighting)** as a lightweight, black-box uncertainty estimator for long-form RAG answers. Unlike SAFE, SEEW does not verify factual correctness; it measures *answer stability under re-sampling* while incorporating retrieved evidence. If the model is confident, repeated generations remain semantically similar; if uncertain, generations diverge into different semantic "modes".

Method. For each query, we generate T samples $\{y_1, \ldots, y_T\}$ via temperature sampling (here T=5, temperature 0.7). We embed each sample with a SentenceTransformer (all-MiniLM-L6-v2) and cluster them using hierarchical clustering with cosine distance, producing semantic clusters C. Uncertainty is captured by the **semantic entropy** over these clusters.

SEEW extends semantic entropy by weighting each sample by its alignment with retrieved passages $\mathcal{D}$. For each y_i, we compute:

$$w_i \;=\; \max_{d \in \mathcal{D}} \cos\big(\phi(y_i), \phi(d)\big), \tag{5}$$

where $\phi(\cdot)$ is the embedding function. Cluster probabilities are then:

$$p(c) \;=\; \frac{\sum_{i:\, y_i \in c} w_i}{\sum_{j=1}^{T} w_j}. \tag{6}$$

Finally, the normalized entropy is:

$$\mathrm{SEEW}(x) \;=\; \frac{-\sum_{c \in C} p(c) \log_2 p(c)}{\log_2 T}, \tag{7}$$

so $\mathrm{SEEW}(x) \in [0, 1]$. Values near 0 indicate one dominant cluster (stable answer), while values near 1 indicate mass spread across clusters (unstable answer).

Interpretation and role in the pipeline. SEEW complements SAFE: SAFE evaluates claim-level correctness, while SEEW flags answer-level instability. Evidence weighting helps suppress off-topic or weakly grounded samples.

Implementation details. We implemented SEEW as a standalone component with resume support and stored results as JSONL (`id`, `SEEW`, `n_clusters`). To keep prompts bounded, each retrieved passage was truncated to a fixed maximum length (e.g., 1200 characters).

Summary. SEEW estimates uncertainty in long-form RAG via multi-sample generation, semantic clustering, and evidence-aware weighting, providing a complementary signal to SAFE for identifying outputs that may need extra verification.

5 Results

The results show that Eccentricity is not performing much better than chance when used as a measure for correctness, which is especially clear in Figure 2. For MARS, the situation is not much better, as shown in Figure 1. The introduced method SEEW also does not perform strongly, with an AUROC of only 0.550. Overall, the uncertainty estimation methods appear to perform worse on the 4-bit quantized model compared to the full-precision model.

Unfortunately, we did not have time to compute a no-RAG baseline for comparison, so we cannot directly isolate whether the weak performance is caused primarily by retrieval quality or by limitations of the uncertainty methods themselves. When looking at individual cases, we found evidence that the issue may partially stem from SAFE rather than the uncertainty estimators. Specifically, among thirty claims that had a normalized truth value above 0.5 (low uncertainty) for both MARS and Eccentricity, but were labeled as incorrect by SAFE on the full model, only 8 claims were actually incorrect upon manual inspection. This implies that at least 22 claims were mislabeled by SAFE, while the uncertainty scores in these cases were consistent with correctness.

Qwen2.5-7B	MARS		Eccentricity		SEEW	
	AUROC	PRR	AUROC	PRR	AUROC	PRR
Full	0.601	0.254	0.532	0.052	0.556	0.084
4bit	0.555	0.206	0.501	-0.022	0.526	0.079

Table 1: Uncertainty estimation performance on long-form RAG.

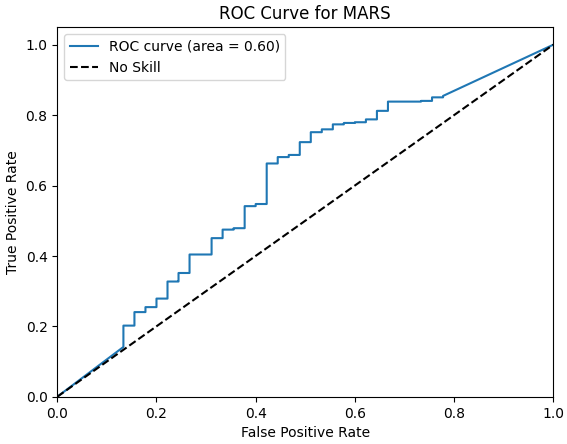

Figure 1: ROC curve for MARS on the full model

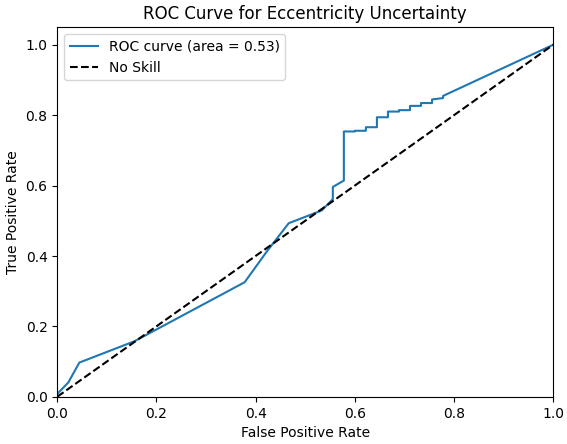

Figure 2: ROC curve for Eccentricity on the full model

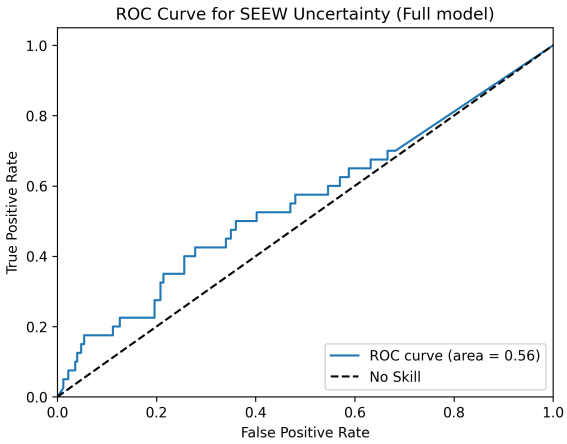

Figure 3: ROC curve for SEEW on the full model

Qwen2.5-7B	Correct Claims	Number of Claims
Full	91.7%	540
4bit	88.9%	524

Table 2: SAFE performance of both full and quantized versions on Factscore-BIO.

6 Conclusion

In this project, we implemented an end-to-end long-form RAG pipeline using BM25 retrieval with cross-encoder reranking and evaluated claim-level factuality with SAFE. We then assessed three uncertainty estimation methods in this setting: the white-box MARS and the black-box ECC and our own proposed SEEW. Overall, the results indicate that these uncertainty signals correlate weakly with long-form correctness in our RAG setup, suggesting that uncertainty estimation remains challenging for long-form, claim-heavy generation even when grounded retrieval is used. However, the problems with SAFE highlight that a setup with, for example, human evaluators might show a different result.

References

[1] Yavuz Faruk Bakman, Duygu Nur Yaldiz, Baturalp Buyukates, Chenyang Tao, Dimitrios Dimitriadis, and Salman Avestimehr. 2024. MARS: Meaning-Aware Response Scoring for Uncertainty Estimation in Generative LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 7752–7767. https://doi.org/10.18653/v1/2024.acl-long.419

[2] Yarin Gal and Zoubin Ghahramani. 2016. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In *international conference on machine learning*. PMLR, 1050–1059.

[3] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. 2023. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. *arXiv preprint arXiv:2302.09664* (2023).

[4] Zhen Lin, Shubhendu Trivedi, and Jimeng Sun. 2024. Generating with Confidence: Uncertainty Quantification for Black-box Large Language Models. *Transactions on Machine Learning Research* (2024). https://openreview.net/forum?id=DWkJCSxKU5

[5] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. 2023. FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. In *EMNLP*. https://arxiv.org/abs/2305.14251

[6] Heydar Soudani, Evangelos Kanoulas, and Faegheh Hasibi. 2025. Why Uncertainty Estimation Methods Fall Short in RAG: An Axiomatic Analysis. In *Findings of the Association for Computational Linguistics: ACL 2025*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 16596–16616. https://doi.org/10.18653/v1/2025.findings-acl.852

[7] Jerry Wei, Chengrun Yang, Xinying Song, Yifeng Lu, Nathan Hu, Jie Huang, Dustin Tran, Daiyi Peng, Ruibo Liu, Da Huang, Cosmo Du, and Quoc V. Le. 2024. Long-form factuality in large language models. In *Proceedings of the 38th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) *(NIPS '24)*. Curran Associates Inc., Red Hook, NY, USA, Article 2567, 72 pages.

[8] Duygu Nur Yaldiz, Yavuz Faruk Bakman, Sungmin Kang, Alperen Öziş, Hayrettin Eren Yildiz, Mitash Ashish Shah, Zhiqi Huang, Anoop Kumar, Alfy Samuel, Daben Liu, Sai Praneeth Karimireddy, and Salman Avestimehr. 2025. TruthTorchLM: A Comprehensive Library for Predicting Truthfulness in LLM Outputs. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, Ivan Habernal, Peter Schulam, and Jörg Tiedemann (Eds.). Association for Computational Linguistics, Suzhou, China, 717–728. https://aclanthology.org/2025.emnlp-demos.54/

[9] Yukun Zhao, Lingyong Yan, Weiwei Sun, Guoliang Xing, Chong Meng, Shuaiqiang Wang, Zhicong Cheng, Zhaochun Ren, and Dawei Yin. 2024. Knowing what llms do not know: A simple yet effective self-detection method. In *Proceedings of the 2024 conference of the north American chapter of the Association for Computational Linguistics: Human language technologies (Volume 1: Long Papers)*. 7051–7063.